

Introduction

This task requires our five-member group to design a peak detector on FPGA, in which two component, command processor and data processor, are involved. Generally, this system keeps receiving and echoing the typed command from external computer via UART. Based on the input stream, the peak detector will identify the peak value among the generated data sequence. After processing, the corresponding results will be returned by the transmitter on FPGA. In detail, the whole system is triggered once a sequence of ANNN, where N denotes a number ranging from 0 to 9, is captured by FPGA. Subsequently, a total of NNN signed numbers are retrieved from the data generator provided. The data processor is going to process the generated data, determine the peak value and its index, together with the three preceding and following numbers. After the results are sent to the command processor, the total seven numbers surrounding the peak will be returned by the L or l command, meanwhile, a command of P or p will display the peak byte and its position. Additionally, this FPGA system is expected to work in a same way as above if another ANNN is typed. It should be noted that the reset is synchronous.

Group Composition, Task Division and Member Contributions

The task division is illustrated by the work breakdown structure in *Figure 1*. The main VHDL development process is divided into four stages involving algorithm level, RTL, simulation and finally testing on board. The report and video tasks are also included in the WBS as final deliverables.

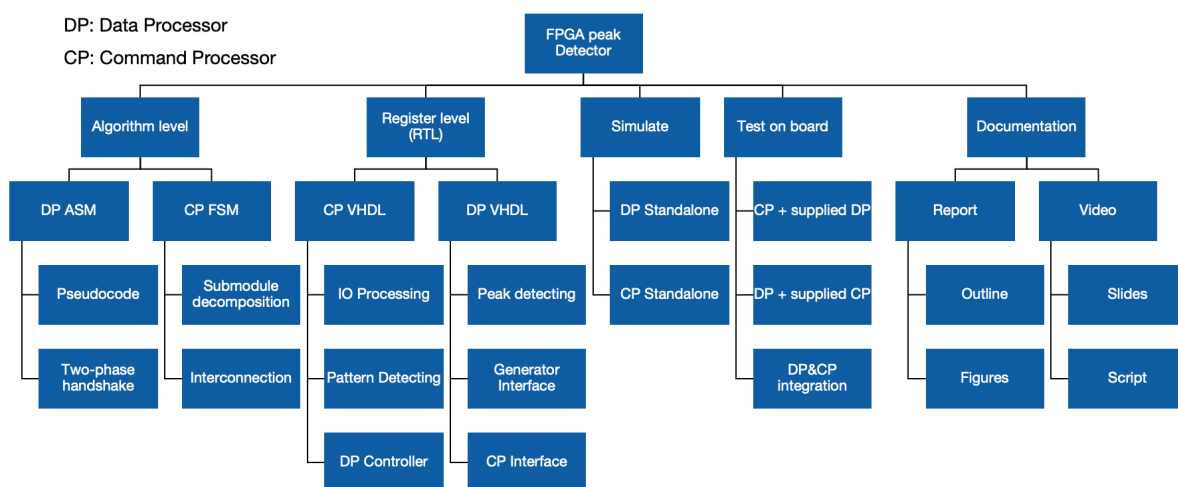


Figure 1: Work Breakdown Structure

The task allocation is mainly based on the WBS chart above. It cannot fully reflect the actual workflow and therefore some other items, like code review and team management, are also added, resulting in the upcoming list of member contributions.

- Yuan Ren (Data Processor Team)
 - Lead
 - VHDL implementation
 - Code review
 - Report documentation
 - Support
 - Simulation and debugging
- Julia Yang (Data Processor Team)
 - Lead
 - FSM/ASM design
 - Simulation and debugging
 - Report Writing (DP part)
 - Support
 - VHDL code review
 - System integration on board
- Xinrui Zhu (Command Processor Team)
 - Lead
 - System architecture design
 - Core FSM/ASM design
 - System integration on board
 - Meeting organisation
 - Support
 - VHDL implementation
 - Code review
- Yuhao Jiang (Command Processor Team)
 - Lead
 - IO controller design
 - VHDL implementation
 - Support
 - Simulation and debugging
 - Report documentation (CP part)
- Xi Zhao (Command Processor Team)
 - Lead
 - Simulation and debugging
 - Report Writing (CP part)
 - Progress tracking
 - Support
 - Code review

Group Project Plan

A timeline is assigned to the stage deliverables indicated above, forming a Gantt Chart shown in *Figure 2*. It is notable that the command processor and data processor are developed and simulated in parallel. Additionally, sufficient time is reserved for simulation and debugging.

Gantt Chart

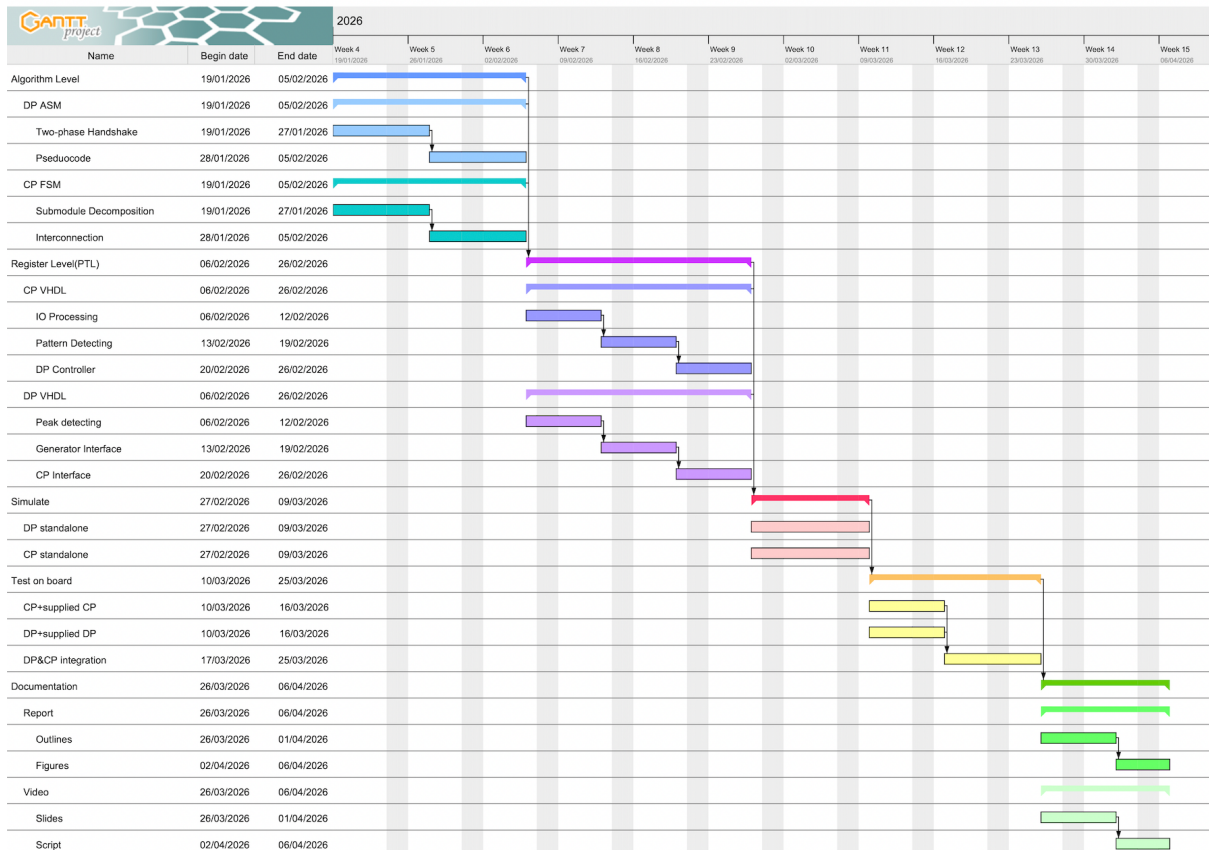


Figure 2: Gantt Chart

Architecture of Data Processor

Based on group number, the data processor is going to find the biggest value within the stream of 8-bit signed numbers generated.

Regarding the interface to the data generator, the data signal is used to receive the data from generator. Additionally, the approach of two-phase handshake, including two control signals, `ctrlIn` and `ctrlOut`, is used to control the behaviour of data generator. Specifically, the request of new data is initialised by toggling the `ctrlOut` signal. Afterwards, the flip of the input `ctrlIn` denotes that the information on the port of data, is valid.

In terms of the connection with the command processor, the start signal decides whether the data processor should continue working or not. The user's input NNN is received from the port of `numWords`. The signals of 8-bit `byte`, 12-bit `maxIndex` and 56-bit `dataResults`, are correspondingly utilised to transmit the generated data, BCD encoded index of peak, and the surrounding seven numbers. The `dataReady` and `seqDone` signals represent if this result above is prepared or not.

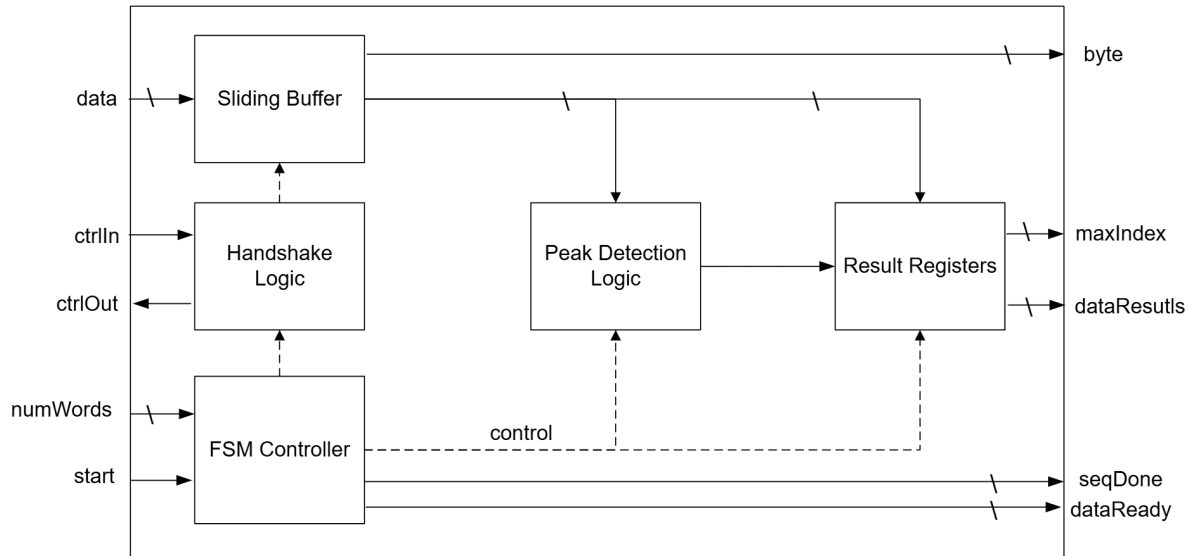


Figure 3: Block-level diagram of data processor

To achieve the desired function, a total of 151 registers as flip flop are applied. Beyond the state registers, a 'finished' register with the default value of 1, and a 'counter' register that augments after a new data is consumed, are used to control the iteration, and the default value is 1. To prevent potential data loss, the 3-digit BCD 'numWords' is sliced, then concatenated and the result is stored in a separated 'NNN' register.

Additionally, the result information mentioned above is registered as well. To monitor the flip of the 'ctrlIn' signal, its delayed version is recorded to compare with the current one.

To remember the preceding and following three numbers, a shifting buffer consisting of seven 8-bit elements is declared. It is left shifted by one position when a new byte is received and appended subsequently. The currently stored max value is compared to the centre value of buffer, namely, buffer[3], and updated if the latter one is greater. Furthermore, the output 'byte' points to the last entry of this shifting buffer.

Therefore, in the first three retrieving cycles, the buffer is being filled but no comparison is performed. Subsequently, NNN-3 data are retrieved and appended in the buffer. Finally, a dummy value, 0X00, is inserted for three times to ensure the last generated number is shifted to the centre position. The output 'byte' points to the last entry of this shifting buffer.

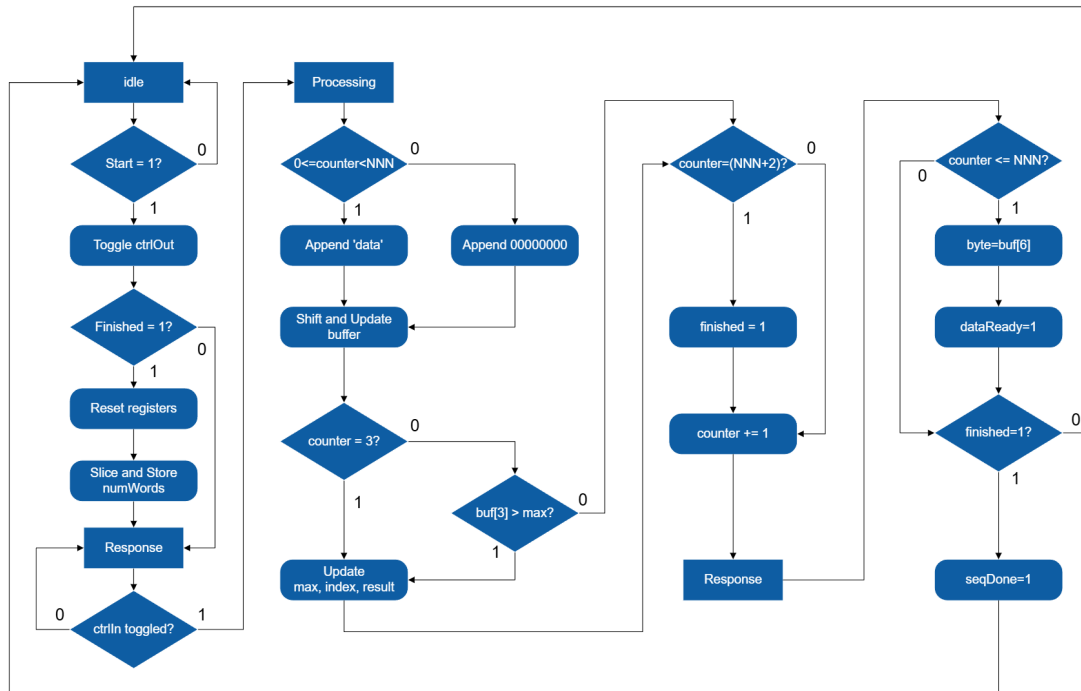


Figure 4: ASM chart of data processor

The procedure above is illustrated by the algorithm state machine in *Figure 4*. which also demonstrate the control of input and output signals apart from the main logic. Initially, the system is activated if the start signal is pulled up. The `ctrlOut` signal is toggled in the same state rather than the next one, to increase the throughput. The state is transited to `processing` once the response of data generator is detected. After processing the new data, the `dataReady` and `seqDone` signals are correspondingly pulled up based on the value of the counter.

Regarding the challenges, some issues were inspected before synthesis. For instance, the buffer was updated, and its new centre value was subsequently employed for comparison in the same process, which may result in combinational glitches in simulation. Thus, some variables that update immediately were designed to store the intermediate data used later.

At the synthesis stage, the combinational loop issue was encountered. To locate the problematic signal, the generated schematic and netlist were checked. It is founded that the `max\_next` signal was assigned when a condition involving itself was satisfied. To solve it, the register `max` signal replaced this combinational next value in judgment. Moreover, the problem of off-by-one was also identified and addressed by correcting the boundary condition.

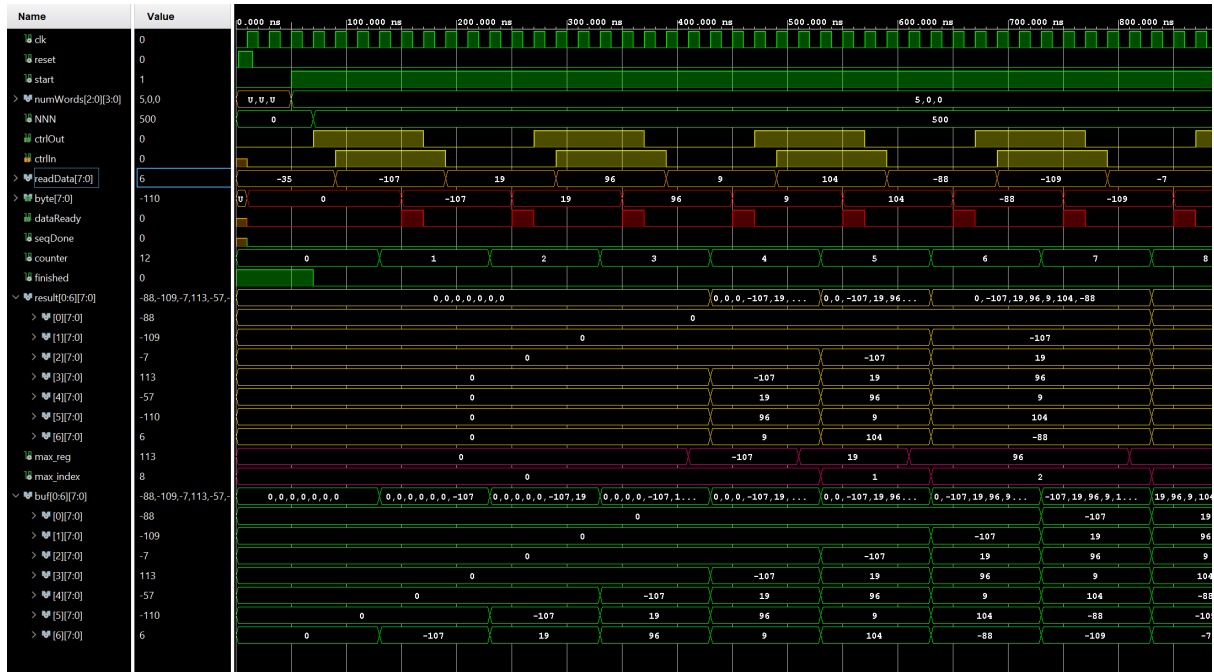


Figure 5: Data processor simulation waveform

The simulation result of data processor is shown in *Figure 5*. The `ctrlOut` signal is flipped when requesting a new data, which is further processed once `ctrlIn` is toggled. It is confirmed that the buffer is updated as expected and its centre value is compared with the stored max one. In conclusion, the data processor behaves as intended in this standalone simulation.

Architecture of Command Processor

In general, the command processor aims to receive the command, echo and return the results to transmitter, trigger the data processor if ANNN sequence is captured, and deactivate it while the data is being transmitted on UART. Beyond the data bus, the receiver interacts with the command processor throughout the `valid` and `done` signals. The former one is pulled up if the input data is ready to be consumed. Simultaneously, a high voltage level on the latter signal indicates the successful capture of input command. The data exchange with receiver follows the same approach and all the IO control logic above is separated into the submodules of `rx_controller` and `tx_controller`. Considering the probability that the echoing function fails when the transmitter is busy, in our design, a ring buffer and dispatcher are introduced to decouple the TX and RX, as shown in *Figure 6*. So, the incoming characters are buffered instead of echoed immediately, preventing data loss and enhancing the system robustness.

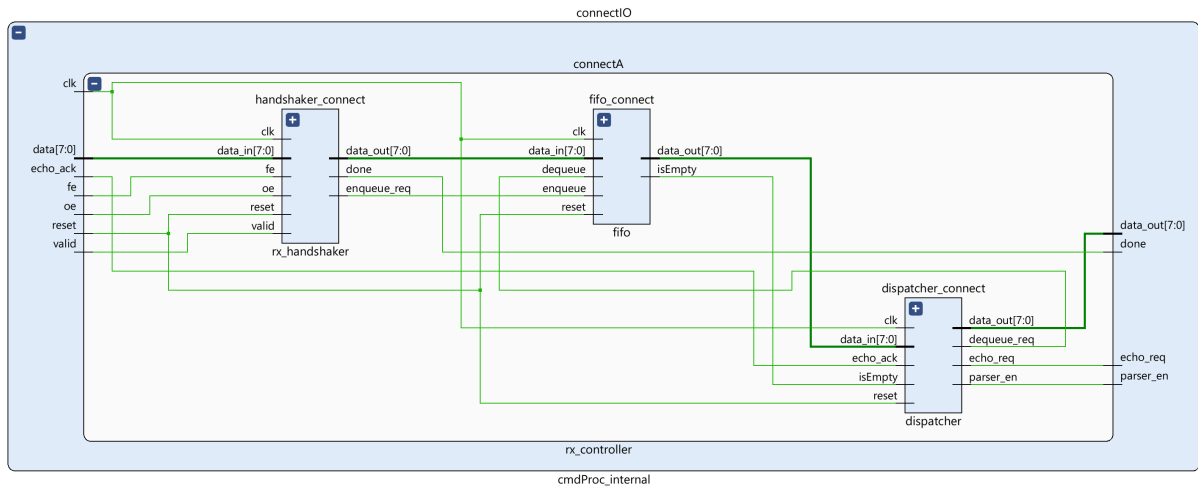


Figure 6: Block-level diagram of RX controller

In terms of the datapath, the received command is sent to the parser and transmitter for further ANNN/L/P judgement and echoing. The parser is in essence a pattern recogniser and accordingly enables the downstream scheduler by issuing the sequence-detected indicators for one cycle. The interconnection of submodules is illustrated in Figure 7.

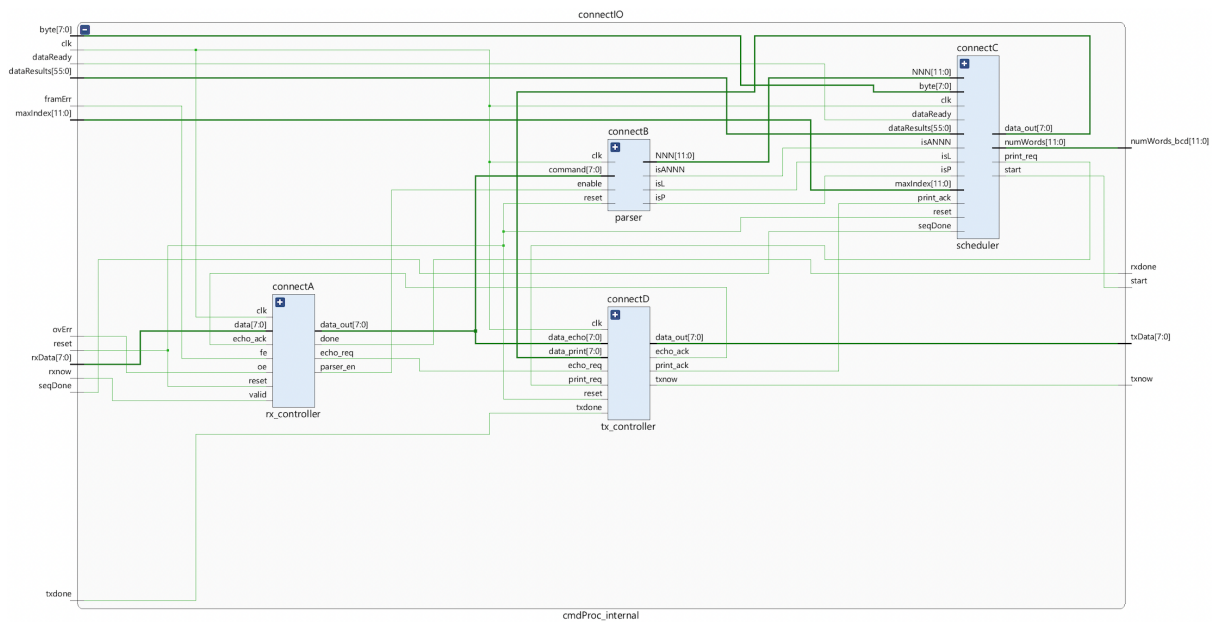


Figure 7: Block-level diagram of overall command processor

To control the printing process, the communications, involving the scheduler-Tx and RX-TX echoing, are based on request-acknowledge handshake. Specifically, the scheduler or dispatcher keeps the request signal high until an acknowledgement is received. The Tx controller will start processing and asserts the acknowledged signal for one cycle once the request is detected. Furthermore, the echoing operation is prior to the result printing.

The scheduler is the most significant one, responsible for the coordination with data processor and TX controller. As for its register usage, since the duration of input signals to be valid is not specified, it is worthwhile using some registers, including ``data``, ``dataResults_reg``, ``maxIndex_reg`` and ``NNN_reg`` to retain the content as their name suggest, avoiding potential loss. Moreover, another ``finished`` register holds the ``seqDone`` input to eliminate the uncertainty of its arrival time.

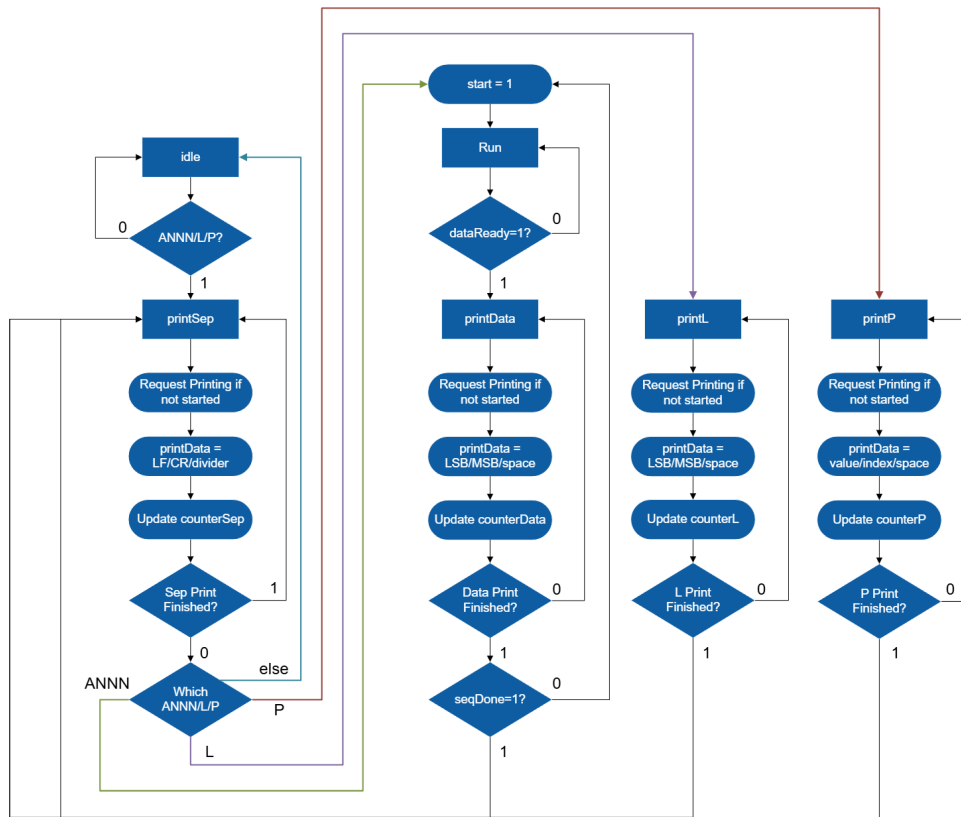


Figure 8: ASM of command processor

Based on the ASM chart of scheduler in *Figure 8*, the workflow is triggered if any of the defined command is recognised. Before any further operations, it is essential to print a delimiter, including CF, LR, in advance.

For ANNN command, the system is going to enter the `run` state, asserting the start output and waiting for the response of data processor. Upon detection of `dataReady`, the 8-bit number on the `byte` signal is printed. Afterwards, whether the scheduler repeats a new cycle of processing or prints another delimiter, depends on whether the `seqDone` has been observed or not. The respective result will be sent out if P or L command is received. Notably, a proper type conversion should be carried out because the results from the data processor are binary encoded, while it is the ASCII code that can be interpreted by PuTTY.

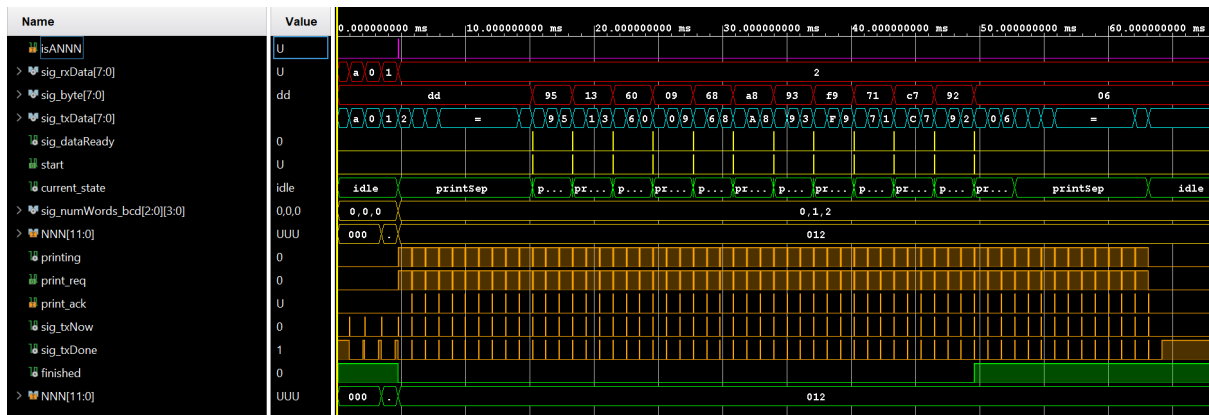


Figure 9: Command processor simulation waveform

The simulation result is illustrated in *Figure 9*. At the beginning, the a012 sequence is successfully identified and trigs the internal processing. After a delimiter is sent out, the `start` signal is asserted to enable the data processor. In general, it is observed that a total of 12 numbers are successfully processed then printed. In summary, it is confirmed that the command process works as expected.

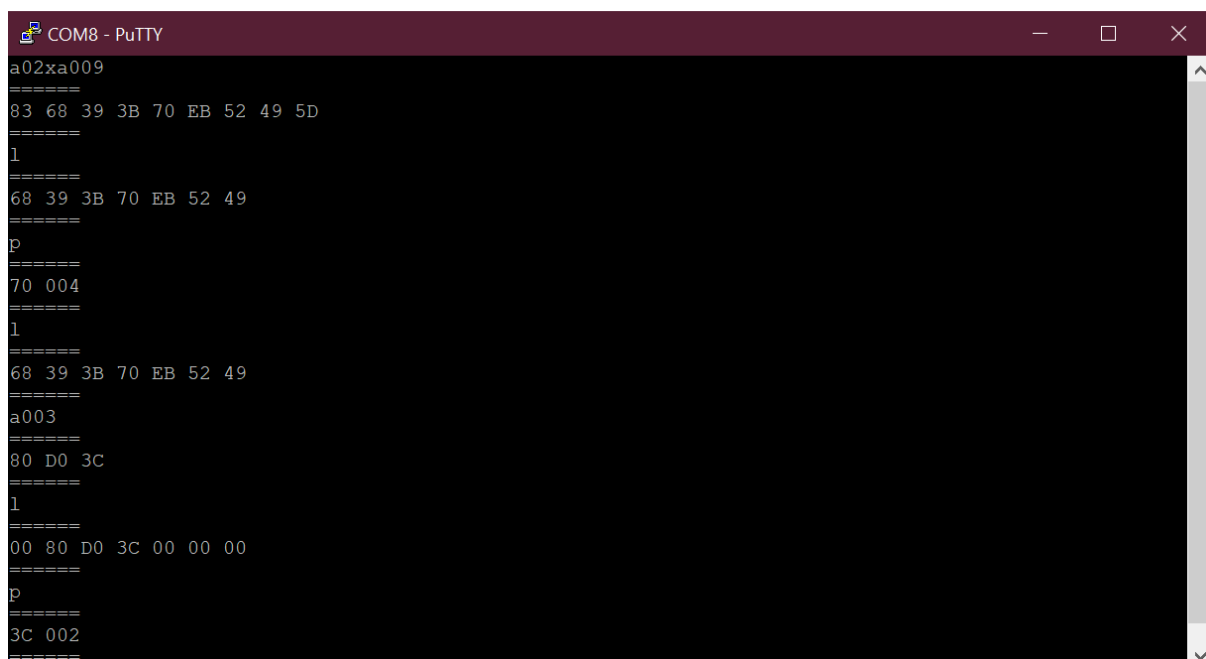
When designing the scheduler, an initial difficulty was the type mismatch between our defined ports and the testbench from university. As a result, a wrapper was utilised to perform the type conversion and there would be no need to modify the entire code. In addition, another issue encountered during synthesis was that the Xilinx Vivado message panel reports the existence of combinational loop. In the end, it was eventually traced back to the cross-module combination loop in the handshake logic. Consequently, a register was added in the handshake path so that the risk of instability was eliminated.

Integration and System Evaluation

In addition to the standalone simulation mentioned above, the joint testing against the reference modules is also conducted for each individual module before integration. At this stage, when verifying the command processor, it is observed that only the output of L is incorrect, and the wrong pattern is unpredictable. To identify the error, some probe signals are temporarily used, which eventually reveals that it is caused by an incorrect boundary while slicing the 56-bit result vector.

In terms of the data processor, several problems emerged while testing on board. The most significant one was that the printed LSB of the last number is always 0, which differs from the actual one. The reason is that the `byte` signal is prematurely cleared before the `seqDone` is asserted. As a result, the `byte` signal is registered rather than directly driven voltage level one, to hold the value until a new turn of processing.

Once these issues are corrected at early stage, there is no problem encountered during the final system integration. The verification on board indicates that the system works as expected, shown in *Figure 10*.



```
COM8 - PuTTY
a02xa009
=====
83 68 39 3B 70 EB 52 49 5D
=====
1
=====
68 39 3B 70 EB 52 49
=====
p
=====
70 004
=====
1
=====
68 39 3B 70 EB 52 49
=====
a003
=====
80 D0 3C
=====
1
=====
00 80 D0 3C 00 00 00
=====
p
=====
3C 002
=====
```

Figure 10: Verification on board

It is worthwhile evaluating the entire system from the perspective of both latency and resource usage. But it cannot fully reflect the performance of custom data processor and command processor since the supplied modules are also involved. Therefore, more focused assessments are also needed. The analysis is visualised by *Table 1* below:

Design Scope	Metrics					
	WNS (ns)	WHS (ns)	Register as Flip Flops	Latch	LUT	IO
Data Processor	+4.31	+0.19	159	0	139	103
Command Processor	+4.54	+0.14	216	0	171	115
Integrated System	+3.37	+0.02	484	0	426	4

Table 1: System Evaluation

The WNS and WHS denote the setup and hold time slack for the register, provided by the timing analysis tool in Vivado. The reference clock period is defined as $10ns$ by either timing constraint file or the generated one by IP catalog. The timing optimisation for both module is applied to decrease the latency, including expanding if-elsif logic to avoid cascaded LUT logic, specifying the range of integer type to decrease fan in and fan out.

According to these results, the estimated maximum frequencies are respectively $175MHz$ for DP, $183MHz$ for CP and $151MHz$ for the entire system, which exceed the task requirement of $100MHz$. In terms of resources usage, there is no latch used in this synchronous design task. The number of flip flops, LUTs are recorded in *Table 1* as well.

Peer Assessment

Group member	1	2	3	4	5
Name	Yuan Ren	Xinrui Zhu	Xi Zhao	Julia Yang	Yuhao Jiang
Username	cz23114	rg23477	me23073	pb24182	hc23104
Leadership	5	5	5	5	5
Team engagement	5	5	5	5	5
Carrying out technical work	5	5	5	5	5
Contributing to the report	5	5	5	5	5
Total/member	20	20	20	20	20

Table 2: Peer Assessment form